



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
ICS2121 – MÉTODOS DE OPTIMIZACIÓN

Apuntes de Métodos de Optimización

*Por Francisco García Aubert y Jorge Vera Andreo
Material en desarrollo. No compartir fuera del curso.
(versión 7, Agosto 2025)*



Índice general

9. Algoritmos de Punto Interior	3
9.1. El algoritmo de Barrera	3
9.2. Convergencia del algoritmo y complejidad	6
9.3. Los métodos interiores prácticos	6
9.4. Algoritmos interiores: el caso general no lineal	7
9.5. Convergencia del algoritmo	8
9.6. Elementos básicos de Optimización Cónica	8
9.7. Ejemplos de problemas cónicos	9
9.7.1. Ejemplo 1: proyección de un punto en un poliedro	9
9.7.2. Ejemplo 2: Support Vector Machine (SVM)	10
9.7.3. Ejemplo 3: Cono de matrices semidefinidas positivas	13
9.7.4. Ejemplo 4: Restricciones cuadráticas	14
9.7.5. Ejemplo 5: Distancias con normas “no euclidianas”	15
9.7.6. Ejemplo 6: Desigualdades lineales de matrices (Linear Matrix Inequalities)	16
9.7.7. Ejemplo 7: Problema cuadrático	16
9.7.8. Ejemplo 8: Otro problema cuadrático	17
9.7.9. Ejemplo 9: Restricciones hiperbólicas	18
9.7.10. Ejemplo 9: Productos de variables no negativas	19

Capítulo 9

Algoritmos de Punto Interior

Los Algoritmos de Punto Interior son uno de los desarrollos más importantes en optimización, tanto desde un punto de vista teórico como práctico. En este capítulo revisaremos los conceptos fundamentales y la forma en que se usan. En sus orígenes, los algoritmos de punto interior fueron desarrollos para abordar problemas de programación lineal pero fueron extendidos al caso no lineal, convexo, de una manera tal que la eficiencia se mantiene muy alta. Esto abrió enormes oportunidades al poder abordarse modelos de optimización que, hasta esos momentos, no podían ser resueltos en forma eficiente. Si bien los desarrollos en este tema ocurrieron entre la segunda mitad de los 1980 y durante la década de 1990, estos mantienen su vigencia hasta el día de hoy gracias a implementaciones computacional muy eficientes y disponibles en el software habitual.

9.1. El algoritmo de Barrera

Para introducir el concepto, consideremos el siguiente problema de Programación Lineal:

$$\begin{aligned} z^* &= \min c^T x \\ \text{s.a. } Ax &\leq b \end{aligned}$$

donde A es $m \times n$ y de rango completo. Sean $\alpha_1, \dots, \alpha_m$ las filas de A . Sea x^* solución óptima.

Como sabemos muy bien, este problema puede ser resuelto mediante el algoritmo Simplex. Sin embargo, este algoritmo se mueve por los vértices del poliedro definido por $Ax \leq b$, es decir, por la frontera. Suena muy natural tratar de avanzar por un “atajo”, es decir, por el interior del poliedro. Necesitamos operacionalizar esto y para eso vamos a construir una función matemática que nos garantice estar en el interior y nos aleje de la frontera del poliedro. Para hacer esto vamos a definir una “función de barrera” (logarítmica) para movernos dentro del poliedro. Sea

$$\Phi(x) = - \sum_{i=1}^m \log(b_i - \alpha_i^T x)$$

Podemos ver que la función evalúa la holgura de un punto x con respecto a las restricciones, pero asumiendo que estas se cumplen. Claramente no es posible cumplir alguna de las desigualdades con igualdad ya que, en ese caso, la función queda indefinida. Por otro lado, si el punto x está en el interior del poliedro, pero muy cerca de la frontera, es decir, que para algún índice i , $b_i \approx \alpha_i^T x$, entonces sucede que:

$$\lim_{b_i \rightarrow \alpha_i^T x} -\log(b_i - \alpha_i^T x) \rightarrow \lim_{\delta \rightarrow 0} -\log(\delta) \rightarrow \infty$$

En otras palabras, la función está definida sólo en el interior del poliedro definido por $Ax \leq b$ y cuando el punto x se aproxima a la frontera, es decir, a las “caras” del poliedro desde adentro, los valores de Φ se hacen cada vez más grandes. De este modo, Φ actúa como “barrera”: no podemos salir del poliedro y resulta muy malo (en términos de valores altos de Φ) estar cerca de la frontera.

Ahora vamos a usar la función de barrera junto con el problema de optimización original. Sea $\mu > 0$, Se define el siguiente problema:

$$P_\mu) \quad \begin{array}{ll} \min & c^T x + \mu \Phi(x) \\ \text{s.a.} & Ax < b \end{array}$$

La primera observación que haremos es que no se necesitan las restricciones, pues $Ax \leq b$ se va a cumplir debido a que la misma barrera logarítmica no va a permitir que se esté afuera de este poliedro. Entonces, el problema es, realmente:

$$P_\mu) \quad \min c^T x + \mu \Phi(x)$$

Notemos que si μ es muy grande, en términos de escala relativa, $\mu \Phi(x)$ es el término principal en la función objetivo, por lo tanto el problema es, básicamente, minimizar la función Φ . Esto dará como resultado un punto bastante “al medio” del poliedro.

Por otro lado, si μ toma un valor ya no tan grande, la información de la función objetivo original, $c^T x$, comienza a tomar peso frente a $\mu \Phi(x)$. La solución, en ese caso, estará “menos al medio” y posicionada más en dirección a donde está la solución óptima del problema.

Si μ es muy chico, el peso de la barrera será mucho menor y hará que la solución del problema esté muy cerca del óptimo x^* . Notemos, sin embargo, que aunque esté disminuida, la barrera está presente y nos impedirá tocar la frontera del poliedro. La figura ilustra las distintas soluciones que se obtienen según el valor de μ .

La discusión anterior sugiere el algoritmo que presentamos a continuación. Para lo que sigue, vamos a definir:

$$f(x, \mu) = c^T x + \mu \Phi(x)$$

Algoritmo de Barrera Logarítmica

Data: $x^0 : Ax^0 < b, \mu_0 > 0$.

while *Criterio de parada no se satisface* **do:**

Resolver aproximadamente $\min_{x \in \mathbb{R}^n} f(x, \mu_k)$ usando x^k como punto de partida.

Sea x^{k+1} esa solución aproximada.

Actualizar μ_k a $\mu_{k+1} < \mu_k$.

$k \leftarrow k + 1$

end

El algoritmo presentado requiere, por supuesto, un punto de partida. Más adelante discutiremos cómo obtenerlo. También comentaremos sobre el criterio de parada. Notemos que el problema $\min_{x \in \mathbb{R}^n} f(x, \mu_k)$ se resuelve en forma aproximada. Eso tiene que ser así, habitualmente nunca podremos encontrar una solución exacta dada la no linealidad de la función de barrera. Un método iterativo tendrá que ser usado para eso y tendrá que usar un punto de partida. Por eso se establece usar la iteración actual como punto de partida.

Notemos que la lógica del algoritmo se basa en la discusión anterior: se propone partir de algún valor para el parámetro μ u sucesivamente disminuirlo para que el término $c^T x$ vaya tomando más importancia y la solución que se obtenga vaya aproximándose a x^* .

Una forma de ir definiendo la sucesión de parámetros de penalización μ_k es $\mu_{k+1} = \beta \mu_k, 0 < \beta < 1$, con $\mu_0 > 0$ dado.

Ahora concentrémonos en la forma de resolver el problema P_{μ_k} .

$$P_{\mu_k}) \quad \min_{x \in \mathbb{R}^n} \{c^T x + \mu_k \Phi(x)\}$$

Es fácil ver que la función $c^T x + \mu_k \Phi(x)$ es estrictamente convexa y tiene todas las propiedades necesarias para resolverlas con la batería de herramientas mostradas en los capítulos anteriores. Particularmente, para resolver el problema penalizado usaremos el método de Newton, para lo cual necesitamos calcular tanto el gradiente como el Hessiano de f .

Derivando la función objetivo tenemos:

$$\begin{aligned}\frac{\partial}{\partial x_j} f(x, \mu) &= c_j + \mu \frac{\partial}{\partial x_j} \Phi(x) \\ &= c_j + \mu \sum_{i=1}^m \frac{\alpha_{ij}}{b_i - \alpha_i^T x}\end{aligned}$$

Sea $e = [1, 1, \dots, 1]^T \in \mathbb{R}^m$ y

$$D(x) = \text{diag}(b_1 - \alpha_1^T x, \dots, b_m - \alpha_m^T x)$$

donde $\text{diag}(u_1, \dots, u_n)$ una matrix diagonal con $u_1, \dots, u_n$ en la diagonal.

Entonces el gradiente de la función objetivo, lo podemos escribir como:

$$\nabla f(x, \mu) = c + \mu A^T D(x)^{-1} e$$

De forma similar se calcula el Hessiano:

$$\nabla^2 f(x, \mu) = \mu A^T D(x)^{-2} A$$

La dirección de Newton es $d = -[\nabla^2 f(x, \mu)]^{-1} \nabla f(x, \mu)$. En este caso:

$$d = -\frac{1}{\mu} [A^T D(x)^{-2} A]^{-1} (c + \mu A^T D(x)^{-1} e)$$

Con esta dirección usamos el método de Newton para resolver aproximadamente P_μ .

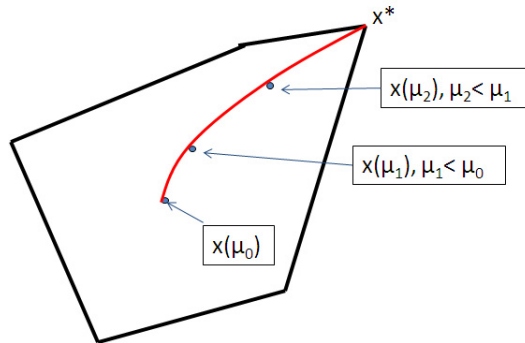
De esta manera, hemos definido un algoritmo general que resuelve una sucesión de problemas penalizados generando una secuencia de puntos que tratan de seguir una “trayectoria central”. La Trayectoria Central del poliedro es el conjunto de soluciones óptimas del problema

$$P_\mu) \min_{x \in \mathbb{R}^n} \{c^T x + \mu \Phi(x)\}$$

cuando μ varía. Mas específicamente, denotemos por $x(\mu)$ la solución óptima de ese problema, para un μ dado. La trayectoria central es el conjunto

$$CP = \{x(\mu) : \mu > 0\}$$

Esta conjunto corresponde a la trayectoria indicada por la línea roja en la figura.



Como no podemos resolver el problema P_μ) en forma exacta, sólo seguimos la trayectoria en forma aproximada. Si se eligen adecuadamente el valor inicial de μ así como la tasa a la cual decrece, β , es posible hacer que este algoritmo funcione en forma muy eficiente, como indicamos a continuación.

9.2. Convergencia del algoritmo y complejidad

Como indicábamos, el parámetro de penalización se cambia como $\mu_{k+1} = \beta\mu_k$, $0 < \beta < 1$ adecuado. El análisis teórico detallado permite concluir que **basta hacer una sola iteración del Método de Newton** para mantener la cercanía adecuada a la trayectoria central. En particular, si se quiere encontrar una solución con una diferencia de a lo más $\epsilon > 0$ con el valor óptimo, se puede mostrar que basta:

$$k = \mathcal{O}\left(\sqrt{m} \log \frac{1}{\epsilon}\right)$$

iteraciones principales para lograr ese objetivo.

¿Y cuánto es el costo por iteración? Volviendo a las fórmulas del gradiente y, especialmente, el Hessiano, vemos que en cada iteración es necesario construir la matriz $A^T D(x)^{-2} A$ y calcular su inversa. Esta matriz cambia completamente en cada iteración, así que el costo numérico debe invertirse en cada iteración principal y es $\mathcal{O}(m^2 + n^3)$. Esto puede ser muy grande, si m y n son grandes. Un tema sorprendente, respecto al comportamiento de los algoritmos de punto interior, es que el número total de iteraciones no suele ser excesivamente grande (en general entre 30 y 60).

Desde luego, querríamos comparar esto con lo que hace el Algoritmo Simplex. Para comenzar, es necesario recordar que el Simplex podría llegar a ser, en el peor de los casos, un algoritmo de complejidad exponencial en n , aunque en la práctica eso no suele ocurrir. Aún así, el número de iteraciones del Simplex en un problema grande puede ser realmente enorme. Esto significa que, eventualmente, aún con el alto costo computacional por iteración, un algoritmo de punto interior podría tardar menos tiempo.

9.3. Los métodos interiores prácticos

Lo que hemos comentado en la sección anterior es un método de punto interior en versión "simple". La implementación práctica es algo diferente y la comentamos brevemente ahora. El desarrollo se aplica a un problema en forma estándar:

$$\begin{aligned} z^* = \min \quad & c^T x \\ \text{s.a.} \quad & Ax = b, x \geq 0 \end{aligned}$$

definiremos la siguiente función de barrera para este problema:

$$\Phi(x) = - \sum_{j=1}^n \log(x_j)$$

esta función tendrá las mismas propiedades que la que habíamos mostrado. Acá debe entender el "interior" como el interior relativo a la restricción $Ax = b$. De este modo, nos interesa mantener $x_j > 0$, pero cumplir siempre la restricción de igualdad. El problema penalizado es:

$$\begin{aligned} P_\mu) \quad & \min \quad c^T x + \mu \Phi(x) \\ \text{s.a.} \quad & Ax = b \end{aligned}$$

Vamos a analizar las condiciones de optimalidad de este problema. Para eso tenemos que construir el correspondiente Lagrangiano:

$$L(x, \pi) = c^T x - \mu \sum_{j=1}^n \log(x_j) + \pi^T (Ax - b)$$

Derivando este Lagrangeano se llega a:

$$\nabla_x L(x, \pi) = c - \mu \begin{bmatrix} 1/x_1 \\ \vdots \\ 1/x_n \end{bmatrix} + A^T \pi$$

Luego, las condiciones KKT para este problema son:

$$\begin{aligned} Ax &= b \\ A^T \pi - \mu D(x)^{-1} e &= c \end{aligned}$$

donde $D(x)$ es la matriz diagonal con elementos diagonales iguales a los x_j . Este es un sistema no lineal de ecuaciones pero se puede resolver, en forma aproximada, mediante el método de Newton “en modo ecuación”, o, equivalentemente, planteando el problema:

$$\min_{x, \pi} \left\| \begin{pmatrix} Ax - b \\ A^T \pi - \mu D(x)^{-1} e - c \end{pmatrix} \right\|$$

Lo anterior permite obtener soluciones aproximadas x y π , tanto en el primal como en el dual. El argumento teórico de convergencia permite llegar a la conclusión de que la complejidad del método resulta en

$$O\left(\sqrt{n} \log \frac{1}{\epsilon}\right)$$

Este tipo de algoritmo se conoce como un **método Primal-Dual** ya que obtiene, simultáneamente, soluciones tanto para el problema primal como para el problema dual. Notemos que, dada la forma en que se aproximan las soluciones, en cada iteración tenemos soluciones factibles, tanto en el primal como en el dual, pero no cumplimos con las condiciones de holguras complementarias. Entonces, mediante un argumento de dualidad débil podemos tener una estimación precisa del error en una iteración.

Resolver el problema de esta forma permite un avance más estable de las iteraciones, en comparación con el algoritmo simple básico que introdujimos al comienzo. Esto es, de hecho, el método real que está implementado en software como CPLEX o Gurobi.

9.4. Algoritmos interiores: el caso general no lineal

Consideremos el siguiente problema con un conjunto S , convexo, cerrado y acotado:

$$\begin{array}{ll} \min & c^T x \\ \text{s.a.} & x \in S \end{array}$$

(“int” denota interior) Observemos que siempre podemos asumir la función objetivo lineal. Si no lo es, puede “pasarse” al conjunto S . Vamos a definir lo que es una función de barrera para este caso general. Definimos una función estrictamente convexa $\Phi : \text{int}(S) \rightarrow \mathbb{R}$ tal que $\Phi(x) \rightarrow \infty$ cuando x se acerca a la frontera de S desde adentro. De este modo, le pedimos a esta función Φ propiedades análogas a las que tenía la función de barrera logarítmica en el caso del problema de programación lineal.

Para un valor $\mu > 0$ definimos el problema penalizado:

$$\begin{array}{ll} P_\mu) \min & c^T x + \mu \Phi(x) \\ \text{s.a.} & x \in \text{int}(S) \end{array}$$

Por las características de Φ , podemos obviar la restricción $x \in \text{int}(S)$, tal como en el caso lineal. Sea $f(x, \mu) = c^T x + \mu \Phi(x)$. Suponiendo que podamos calcular $\nabla f(x, \mu)$ y $\nabla^2 f(x, \mu)$, podemos plantear el siguiente algoritmo:

Algoritmo de Barrera General

Data: Sea $x^0 \in \text{int}(S)$, $k = 0$, $\mu_0 > 0$, $0 < \beta < 1$.

while *Criterio de parada no se satisface* **do:**

Usar el método de Newton, con x^k como punto inicial, para resolver aproximadamente P_{μ_k} :

$$\min f(x, \mu_k) = \min \{c^T x + \mu_k \Phi(x)\}$$

sea x^{k+1} la solución aproximada.

$$\mu_{k+1} = \beta \mu_k, k = k + 1.$$

end

Al igual que en el caso lineal, esto trata de seguir una trayectoria central por el interior de S .

La pregunta central aquí es cuál es la función de barrera apropiada, y eso depende de cada conjunto S , pero la elección de la función de barrera, según el conjunto S , debe permitir que el método de Newton funcione eficientemente. Si eso es así, se puede obtener convergencia lineal.

Uno de los desarrollos más significativos en Optimización es de los años 1990 en el que Yuri Nesterov y Arkadi Nemirovski identificaron toda una clase de funciones que pueden usarse de barreras, según el conjunto S , y para las cuales el funcionamiento del algoritmo es eficiente. Esas funciones tienen una propiedad matemática (que no detallaremos) que se conoce como “autoconcordancia” (*self-concordance*). Si vamos a destacar que la clave del análisis es que en todas las iteraciones vamos a medir distancias con una norma especial que se define a partir del Hessiano $H(x, \mu) = \nabla^2 f(x, \mu)$. Se define, entonces, la siguiente norma “local” en x :

$$\|u\|_{H(x, \mu)} = \sqrt{u^T H(x, \mu) u}$$

Esta no es una norma euclidiana (Salvo si H fuera la identidad), y depende del punto x donde estamos. La siguiente propiedad es muy interesante: Sea $x \in \text{int}(S)$ y sea $B_H(x, 1) = \{y : \|x - y\|_{H(x, \mu)} \leq 1\}$, la bola unitaria (en la norma local) en torno a x . Entonces:

$$B_H(x, 1) \subset \text{int}(S), \quad \text{para todo } x \in S$$

Es decir, cualquier bola de radio 1 (en la norma local) está en el interior de S . Esto parece complementario no intuitivo pero hay que recordar que estamos viendo la geometría con una norma que cambia constantemente. Lo que esto hace es introducir una geometría de Riemann, como se les conoce, en el conjunto S y la regularidad que induce esta geometría hace que el Método de Newton tenga el mejor comportamiento posible.

9.5. Convergencia del algoritmo

Usando las propiedades anteriores se puede mostrar que basta una iteración del Método del Newton en el problema P_{μ_k} para tener una solución x^k suficientemente aproximada, medido en la “norma local”. Más aún también se puede mostrar que

$$c^T x^k - z^* \leq \vartheta \mu_k$$

donde ϑ es un parámetro que depende únicamente de las características de la función de barrera. En el caso lineal $Ax \leq b$ con m restricciones, se tiene $\vartheta = m$

Entonces, como $\mu_k = \mu_0 \beta^k$, se tiene:

$$c^T x^k - z^* \leq \vartheta \mu_0 \beta^k$$

Finalmente se deduce que si se quiere encontrar una solución con una diferencia de a lo más $\epsilon > 0$ con el valor óptimo, basta un número de iteraciones del orden de:

$$k = O\left(\sqrt{\vartheta} \log \frac{1}{\epsilon}\right)$$

Entonces, los algoritmos de punto interior conservan, en el caso general, el buen comportamiento que tienen en el caso lineal. Ahora bien, qué tipos de problemas no lineales podrían ser abordados con estos métodos lo presentaremos en la próxima sección.

9.6. Elementos básicos de Optimización Cónica

El siguiente es un problema lineal cónico:

$$\begin{array}{ll} \text{mín} & c^T x \\ \text{s.a.} & Ax = b \\ & x \in K \end{array}$$

donde K es un cono convexo. Recordemos que K es cono convexo si es convexo y $x \in K \Rightarrow \lambda x \in K, \lambda \geq 0$. Los siguientes son ejemplos de conos, de particular importancia:

- $\mathbb{R}_+^n = \{x \in \mathbb{R}^n : x \geq 0\}$. Este es el cono que aparecerá en el problema de programación lineal en forma estándar. Es decir, PL es un caso particular de problema cónico.

- $Q^{n+1} = \{(x, t) : x \in \mathbb{R}^n, t \in \mathbb{R} \text{ y } \|x\|_2 \leq t\}$, el Cono de Segundo Orden o Cono de Lorenz en $n + 1$ dimensiones. Veremos que es particularmente importante para modelar algunos problema no lineales.
- $S_+^{n \times n} = \{X \in \mathbb{R}^{n \times n} : X \text{ es simétrica y semidefinida positiva}\}$. Este cono también es de gran importancia en desarrollo modernos de optimización, mostraremos un ejemplo más adelante.

Miremos bien el cono $Q^{n+1} = \{(x, t) : x \in \mathbb{R}^n, t \in \mathbb{R} \text{ y } \|x\|_2 \leq t\}$, el Cono de Segundo Orden.

$$\|x\|_2 \leq t \iff \sqrt{\sum_{j=1}^n x_j^2} \leq t$$

La forma de este cono es la que muestra la figura:

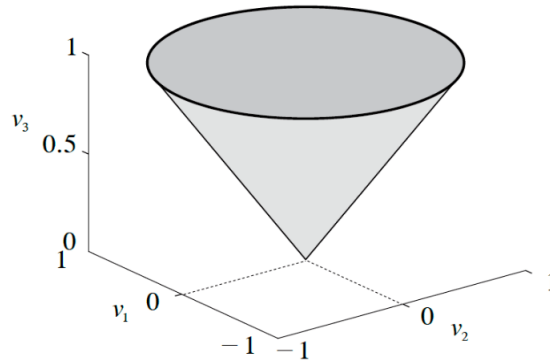


Figura 9.1: Fuente: Static Limit Analysis of Reinforced Soil Structures by a Simple Finite Element and Second-Order Cone Programming, <http://dx.doi.org/10.1590/1679-78253745>, CCBY4.0

Para cada uno de esos conos hay una función de barrera con buenas propiedades. En particular, para el cono $\mathbb{R}_+^n$ la barrera es, simplemente:

$$\Phi(x) = -\sum_{j=1}^n \log x_j$$

Para los otros conos, explicaremos las barreras en los ejemplos. El punto clave acá es que hay problemas que se pueden formatear como problemas de Optimización Cónica.

Cápsula de video 9.6.1: Optimización Cónica

profesor Jorge Vera explica en clases elementos básicos de Optimización Cónica y algunos ejemplos.
<https://youtu.be/3c27xb2zHdE>

9.7. Ejemplos de problemas cónicos

9.7.1. Ejemplo 1: proyección de un punto en un poliedro

Consideremos el poliedro $P = \{x \in \mathbb{R}^n : Ax \leq b\}$. Sea $u \in \mathbb{R}^n$ dado. Queremos encontrar el punto en P más cercano a u en norma euclidiana. Esto se encuentra resolviendo:

$$\begin{array}{ll} \text{mín} & \|x - u\|_2 \\ \text{s.a.} & Ax \leq b \end{array}$$

Esto puede ser resuelto con algún método de primer orden proyectado o con el algoritmo de Frank-Wolfe, pero también se puede usar un algoritmo de Punto Interior, pero para esto hay que hacerle una serie de

transformaciones. El problema anterior es equivalente a:

$$\begin{array}{llll}
 \begin{array}{ll} \text{mín} & t \\ \text{s.a.} & Ax \leq b \\ & \|x - u\|_2 \leq t \end{array} & \begin{array}{l} \text{que} \\ \text{equivale a} \end{array} & \begin{array}{ll} \text{mín} & t \\ \text{s.a.} & Ax \leq b \\ & v = x - u \\ & \|v\|_2 \leq t \end{array} & \begin{array}{l} \|v\|_2 \leq t \text{ es un cono} \\ \text{de segundo orden} \\ \text{entonces se puede} \\ \text{reescribir como} \end{array} & \begin{array}{ll} \text{mín} & t \\ \text{s.a.} & Ax \leq b \\ & v = x - u \\ & (v, t) \in Q^{n+1} \end{array}
 \end{array}$$

Este es un problema lineal con restricciones cónicas: restricciones de Cono de Segundo Orden. Este problema se conoce como SOCP: "Second Order Cone Problem". El problema anterior se resuelve con un algoritmo de punto interior usando la siguiente función de barrera para el cono:

$$\Phi(x, t) = -\log(t^2 - \|v\|_2^2)$$

Esto es como que se tomara la restricción que vincula a las t y a $\|v\|_2$, pero elevado al cuadrado, cosa que es importante ya que eso garantiza que la barrera sea autoconcordante. De este modo, el problema penalizado es:

$$\begin{array}{ll}
 \text{mín} & t - \mu \left(\log(t^2 - \|v\|_2^2) + \sum_{i=1}^m \log(b_i - \alpha_i^T x) \right) \\
 \text{s.a.} & v = x - u
 \end{array}$$

donde $\alpha_1, \dots, \alpha_m$ son las filas de A . Gurobi y CPLEX pueden resolver un problema de esta forma, al igual que otros sistemas de software que implementan algoritmos de punto interior para problemas cónicos.

9.7.2. Ejemplo 2: Support Vector Machine (SVM)

Un problema importante en Machine Learning es el de poder distinguir comportamientos de observaciones. Por ejemplo, supongamos que en un hospital hemos estudiado a personas que llegan al servicio de urgencias y que tienen una serie de síntomas y características, que se miden al ingresar (esto se conoce como "Triage"). Algunas de esas personas han sido atendidas y dadas de alta inmediatamente después, pero otras tuvieron que ser admitidas al hospital después de urgencia. Nos interesa saber si alguien tendrá que ser hospitalizado, para poder preparar unidades médicas, camas, etc. De este modo, el objetivo es: a partir de la información anterior, inferir una "fórmula" que permita anticipar si los nuevos pacientes que llegan a urgencia y sean evaluados en el traje, requerirán o no hospitalización. Supongamos que tenemos:

- n variables que puedo observar y medir.
- Hay una propiedad que quiero discriminar (por ejemplo, "alta" u "hospitalización").
- Tenemos r observaciones de esas n variables y que corresponden a "alta".
- Tenemos q observaciones de esas n variables y que corresponden a "hospitalización".
- Sea $u^1, \dots, u^r$ vectores en $\mathbb{R}^n$ que corresponden a "alta".
- Sea $v^1, \dots, v^q$ vectores en $\mathbb{R}^n$ que corresponden a "hospitalización".

Si pudiéramos visualizar en el espacio de n dimensiones para ver como se comportan los datos. Por ejemplo, si las variables son edad y temperatura corporal, podría tenerse una situación como la de la Figura 9.2.

Se puede notar que existe una línea recta que separa estos dos puntos y sería ideal poder encontrar la ecuación de esta línea de modo que separe lo mejor posible, para así poder discriminar entre todos los datos. Por supuesto, cuando hay muchas variables, este es un problema multidimensional de encontrar un hiperplano que separe dos nubes de puntos. La separación debería ser tal que el separado esté lo más "equidistante" posible de ambas nubes.

Recordemos que un hiperplano tiene esta forma: $H_{\alpha, \beta} = \{x \in \mathbb{R}^n : \alpha^T x = \beta\}$, donde $\alpha \in \mathbb{R}^n, \beta \in \mathbb{R}$. Necesitamos también la distancia de un punto al hiperplano. Si $\bar{x} \in \mathbb{R}^n$, la distancia hasta el hiperplano es:

$$\text{dist}(\bar{x}, H_{\alpha, \beta}) = \frac{|\alpha^T \bar{x} - \beta|}{\|\alpha\|_2}$$

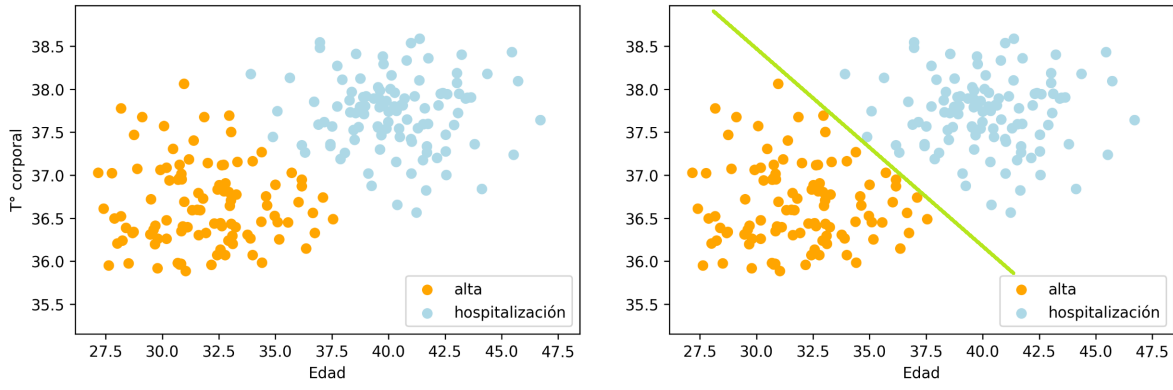


Figura 9.2: Ejemplos donde se distingue un comportamiento en urgencias y se puede establecer un límite separador

Entonces, buscamos un hiperplano $H_{\alpha,\beta}$ que maximice la menor distancia a ambas nubes de puntos. Es decir, queremos resolver el siguiente problema:

$$\begin{aligned} \text{máx } & \gamma \\ \text{s.a. } & \text{dist}(u^i, H_{\alpha,\beta}) \geq \gamma, i = 1, \dots, r \\ & \text{dist}(v^j, H_{\alpha,\beta}) \geq \gamma, j = 1, \dots, q \\ & \alpha \in \mathbb{R}^n, \beta \in \mathbb{R} \end{aligned}$$

El problema anterior, en forma más explícita:

$$\begin{aligned} \text{máx } & \gamma \\ \text{s.a. } & \frac{|\alpha^T u^i - \beta|}{\|\alpha\|_2} \geq \gamma, i = 1, \dots, r \\ & \frac{|\alpha^T v^j - \beta|}{\|\alpha\|_2} \geq \gamma, j = 1, \dots, q \\ & \alpha \in \mathbb{R}^n, \beta \in \mathbb{R} \end{aligned} \quad \begin{aligned} & \text{Imponemos } \|\alpha\|_2 = 1 \\ & \text{para normalizar.} \\ & \text{Así, el problema es:} \end{aligned} \quad \begin{aligned} \text{máx } & \gamma \\ \text{s.a. } & \alpha^T u^i - \beta \geq \gamma, i = 1, \dots, r \\ & \beta - \alpha^T v^j \geq \gamma, j = 1, \dots, q \\ & \|\alpha\|_2 = 1 \end{aligned}$$

Notar que hemos impuesto arbitrariamente un “lado” al hiperplano (y estamos asumiendo que las nubes no se intersectan), por lo que el valor absoluto se toma en el caso que corresponde y en las restricciones 1 y 2 una es el negativo de la otra. Este problema NO es convexo, debido a la restricción $\|\alpha\|_2 = 1$. Pero si dividimos por γ y reemplazamos:

$$z = \frac{\alpha}{\gamma}, \quad \delta = \frac{\beta}{\gamma} \quad \text{Lo que a su vez significa que:} \quad \|z\| = \frac{\|\alpha\|}{\gamma} = \frac{1}{\gamma}$$

De este modo, podemos notar que maximizar γ equivale a minimizar la norma euclidiana de z . Así, el problema equivalente es:

$$\begin{aligned} \text{mín } & \|z\|_2 \\ \text{s.a. } & z^T u^i - \delta \geq 1, i = 1, \dots, r \\ & \delta - z^T v^j \geq 1, j = 1, \dots, q \\ & z \in \mathbb{R}^n, \delta \in \mathbb{R} \end{aligned}$$

Este problema sí es convexo, y puede adaptarse al formato de Cono de Segundo Orden. Así, el problema se transforma en:

$$\begin{aligned} \text{mín } & t \\ \text{s.a. } & z^T u^i - \delta \geq 1, i = 1, \dots, r \\ & \delta - z^T v^j \geq 1, j = 1, \dots, q \\ & \|z\|_2 \leq t \\ & z \in \mathbb{R}^n, \delta \in \mathbb{R}, t \geq 0 \end{aligned} \quad \begin{aligned} & \text{Que es equivalente a:} \end{aligned} \quad \begin{aligned} \text{mín } & t \\ \text{s.a. } & z^T u^i - \delta \geq 1, i = 1, \dots, r \\ & \delta - z^T v^j \geq 1, j = 1, \dots, q \\ & (z, t) \in Q^{n+1} \\ & z \in \mathbb{R}^n, \delta \in \mathbb{R}, t \geq 0 \end{aligned}$$

donde hemos hecho explícita la restricción cónica.

Este problema es importante en Machine Learning donde se conoce como *Support Vector Machine* (SVM) y se usa ampliamente en análisis de datos de gran tamaño.

Si las nubes no están estrictamente separadas, pero aun así se pueden separar en dos *clusters* se busca un rango de hiperplano que vayan por la zona donde se mezclan las nubes (Figura 9.3). Un caso nuevo que está dentro de los dos hiperplanos simplemente es difícil de decidir, se produce una zona. Si la zona es muy ancha, entonces no se puede discriminar tan bien.

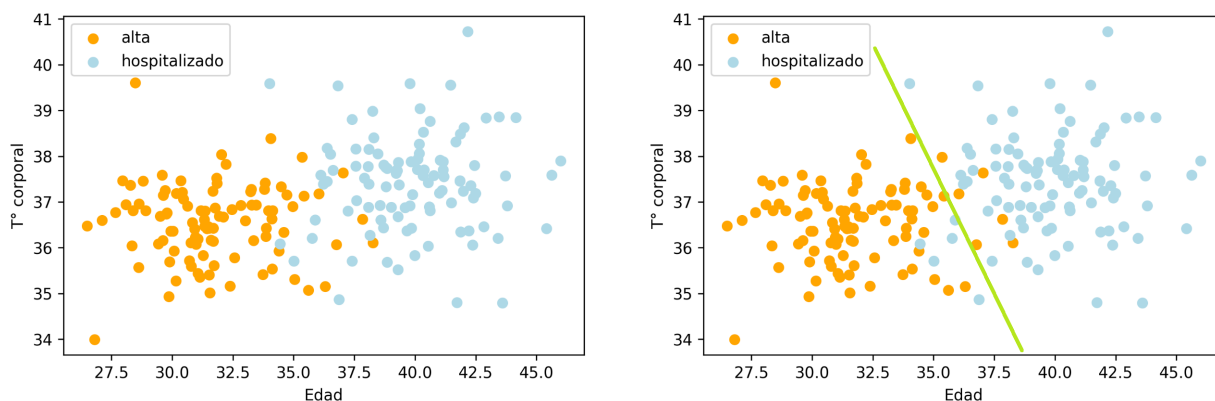


Figura 9.3: Distinción de “hospitalizados” y “alta” por *clusters*

Si la separación no puede ser lineal pero sí “curva”, se le aplica una transformación (llamada Kernel) de los datos de modo de llevarlos a una nueva dimensión donde se puedan separar linealmente. En caso de que una clase de datos se concentren en el centro y la otra afuera, tal como sucede con el primer cuadro de la Figura 9.4, se puede aplicar la función $f(x, y) = x^2 + y^2$, lo que lleva a los puntos más lejanos del centro a estar más “altos” en una nueva tercera dimensión (segundo y tercer gráfico de la Figura 9.4) y ahí se pueden separar con un hiperplano, de modo de obtener finalmente la separación deseada (último cuadro de la Figura 9.4).

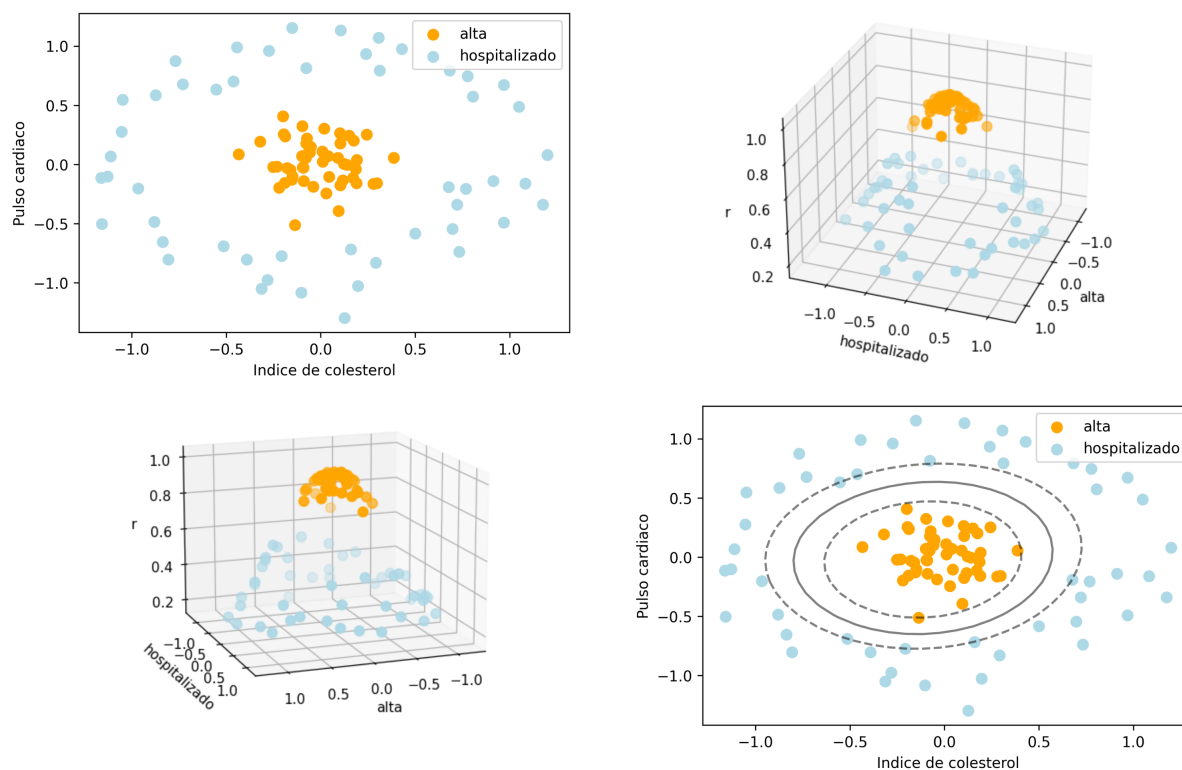


Figura 9.4: Distinción de “hospitalizados” y “alta” por *clusters* no lineales

9.7.3. Ejemplo 3: Cono de matrices semidefinidas positivas

Tengo m puntos en $\mathbb{R}^n, c^1, \dots, c^m$. Estos pueden ser el resultado de muchas observaciones de n variables que interesa estudiar. Queremos saber si hay “tendencias” en los datos: encontrar lo que se conoce como las “Componentes Principales”. Supongamos que tenemos datos que, al graficarlos, se ven como muestra la figura:

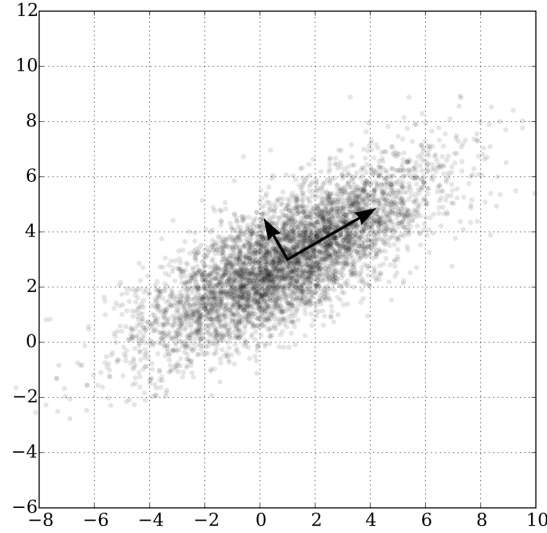


Figura 9.5: Fuente: www.wikipedia.com, CCBY4.0

Claramente hay una correlación entre las dos variables pero si sólo hacemos una regresión lineal, se pierde la información de lo que sucede en la dirección ortogonal. Por otro lado, se puede pensar en buscar la elipse de menor volumen que encierra los puntos. Los ejes principales de esa elipse son las “componentes principales” de los datos. Entonces, el problema que queremos resolver es encontrar un elipsoide de mínimo volumen que contenga todos los puntos. recordemos que un Elipsoide se define como:

$$\text{Ell}(R, z) = \{x \in \mathbb{R}^n : (x - z)^T R (x - z) \leq 1\}$$

queda determinada por el centro z y la matriz R , que tiene que ser simétrica y definida positiva. El problema se puede plantear como:

$\begin{aligned} &\min_{(R, z)} \text{Vol}(\text{Ell}(R, z)) \\ &\text{s.a. } c^i \in \text{Ell}(R, z), i = 1, \dots, m \\ &\quad R \in S_+^{n \times n}, z \in \mathbb{R}^n \end{aligned}$	Que es lo mismo que:	$\begin{aligned} &\min_{(R, z)} \text{Vol}(\text{Ell}(R, z)) \\ &\text{s.a. } (c^i - z)^T R (c^i - z) \leq 1, i = 1, \dots, m \\ &\quad R \in S_+^{n \times n}, z \in \mathbb{R}^n \end{aligned}$
---	---------------------------------	---

Esto último tiene una expresión cuadrática que tiene una forma que puede ser colocada en el formato de cono de segundo orden.

Por otro lado,

$$\text{Vol}(\text{Ell}(R, z)) = \frac{\pi^{n/2}}{\Gamma(\frac{n}{2} + 1)} \det(R)^{-1/2}$$

Entonces, minimizar $\text{Vol}(\text{Ell}(R, z))$ es lo mismo que minimizar $-\log(\det(R))$ (Notar que el resto es una constante) y esa función es fácil de manejar (de hecho, se conocen expresiones para su gradiente y Hessiano, pero no profundizaremos en esto). El problema es, entonces:

$$\begin{aligned} & \min_{(R,z)} -\log(\det(R)) \\ & \text{s.a. } (c^i - z)^T R (c^i - z) \leq 1, i = 1, \dots, m \\ & R \in S_+^{n \times n}, z \in \mathbb{R}^n \end{aligned}$$

Para el cono de matrices semidefinidas, $R \in S_+^{n \times n}$, es habitual usar la notación $R \succeq 0$, así que el problema se puede también escribir como:

$$\begin{aligned} & \min_{(R,z)} -\log(\det(R)) \\ & \text{s.a. } (c^i - z)^T R (c^i - z) \leq 1, i = 1, \dots, m \\ & R \succeq 0, z \in \mathbb{R}^n \end{aligned}$$

Las restricciones $(c^i - z)^T R (c^i - z) \leq 1$ se pueden transformar en cónicas más lineales haciendo: $(v, 1) \in Q^{n+1}$, $v = U(x - c^i)$, donde $U \in \mathbb{R}^{n \times n}$ es tal que $U^T U = R$.

Esto es lo que se conoce como un problema de Programación Semi-definida o SDP (Semi-definite Programming) y puede ser abordado con algoritmos de punto interior. La pregunta es cuál es la barrera que se debe usar para el cono $S_+^{n \times n}$ de matrices simétricas y definidas positivas.

Sean $\lambda_1, \dots, \lambda_n$ los valores propios de R . R es d.p. si y sólo si $\lambda_i > 0, i = 1, \dots, n$. Entonces, la barrera natural es:

$$\Phi(R) = -\sum_{i=1}^n \log \lambda_i = -\log \det(R)$$

Se puede mostrar que esta función Φ es estrictamente convexa y se conocen su gradiente y Hessiano. Además, es una barrera autoconcordante y permite que el algoritmo funcione muy eficientemente.

9.7.4. Ejemplo 4: Restricciones cuadráticas

Supongamos que en un problema de optimización aparece una restricción de la forma

$$x^T Q x + d^T x \leq \alpha$$

donde $Q \in \mathbb{R}^{n \times n}$ es simétrica y definida positiva, $d \in \mathbb{R}^n$ y α es un escalar. En estas condiciones, la restricción anterior define una región convexa en el espacio $\mathbb{R}^n$. Vamos a mostrar como transformar esta expresión en una que permite visualizar restricciones de cono de segundo orden.

Esto se puede hacer considerando que, como Q es definida positiva, existe R , invertible tal que $R^T R = Q$. Este R puede venir, por ejemplo, de la Descomposición de Cholesky, pero también de la diagonalización de la matriz Q . Lo que aquí hay que hacer es “completar el cuadrado” en la expresión de la izquierda. Esto lleva a la siguiente formulación:

$$x^T R x + \frac{1}{4}(1 + d^T x - \alpha)^2 \leq \frac{1}{4}(1 - d^T x + \alpha)^2$$

lo que equivale a:

$$\sqrt{x^T R x + \frac{1}{4}(1 + d^T x - \alpha)^2} \leq \frac{1}{2}(1 - d^T x + \alpha)$$

y que significa que

$$\left(R x, \frac{1}{2}(1 + d^T x - \alpha), \frac{1}{2}(1 - d^T x + \alpha) \right) \in Q^{n+2}$$

que es la expresión de Cono de Segundo Orden que necesitamos, y que se puede poner más simplificada como:

$$\begin{aligned} & ((R x, s), t) \in Q^{n+2} \\ & \frac{1}{2}(1 + d^T x - \alpha) = s \\ & \frac{1}{2}(1 - d^T x + \alpha) = t \end{aligned}$$

9.7.5. Ejemplo 5: Distancias con normas “no euclidianas”

Considere el siguiente problema de optimización, que busca determinar los puntos más cercanos en dos poliedros, pero donde la cercanía se mide en una norma no euclidea (obviamente, si los poliedros se intersectan el valor óptimo es 0):

$$\begin{array}{ll} \text{mín} & \|x - y\|_Q \\ \text{s.a.} & Ax = b \\ & By = d \\ & x \geq 0 \\ & y \geq 0 \end{array}$$

donde $\|u\|_Q = \sqrt{u^T Q u}$, con $Q \in \mathbb{R}^{n \times n}$ simétrica y definida positiva. Además, se tiene que $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $B \in \mathbb{R}^{p \times n}$, $d \in \mathbb{R}^p$. Este problema se puede resolver con algoritmos de punto interior en formulaciones cónicas.

Primero vamos a reformular el problema anterior como otro de optimización lineal cónica (es decir, la función objetivo debe ser lineal y las restricciones lineales y/o cónicas.) Tenemos que el problema equivale a:

$$\begin{array}{ll} \text{mín} & t \\ \text{s.a.} & \|x - y\|_Q \leq t \\ & Ax = b \\ & By = d \\ & x \geq 0 \\ & y \geq 0 \end{array}$$

Ahora, tenemos que $\|x - y\|_Q = \sqrt{u(x - y)^T Q (x - y)}$. Por otro lado, como Q es d.p., se tiene que $Q = M^T M$, donde M es también d.p. Entonces;

$$\|x - y\|_Q = \sqrt{(x - y)^T Q (x - y)} = \sqrt{(x - y)^T M^T M (x - y)} = \|M(x - y)\|_2$$

Luego, la formulación es:

$$\begin{array}{ll} \text{mín} & t \\ \text{s.a.} & \|M(x - y)\|_2 \leq t \\ & Ax = b \\ & By = d \\ & x \geq 0 \\ & y \geq 0 \end{array}$$

que es lo mismo que:

$$\begin{array}{ll} \text{mín} & t \\ \text{s.a.} & Ax = b \\ & By = d \\ & x \geq 0 \\ & y \geq 0 \\ & (M(x - y), t) \in K^{n+1} \end{array}$$

donde K^{n+1} denota el cono de segundo orden. Una formulación más “limpia” sería la siguiente:

$$\begin{array}{ll} \text{mín} & t \\ \text{s.a.} & Ax = b \\ & By = d \\ & z = M(x - y) \\ & x \geq 0 \\ & y \geq 0 \\ & (z, t) \in K^{n+1} \end{array}$$

¿Cuál será la barrera que se debe usar? La barrera se aplica a las desigualdades y conos, en este caso es:

$$\Phi(x) = - \left\{ \log(t^2 - z^T z) + \sum_{j=1}^n \log x_j + \sum_{j=1}^n \log y_j \right\}$$

9.7.6. Ejemplo 6: Desigualdades lineales de matrices (Linear Matrix Inequalities)

Sean $A_0, A_1, \dots, A_r$ matrices de $\mathbb{R}^{p \times p}$ simétricas (sólo simétricas). Definamos

$$B(x) = A_0 + \sum_{i=1}^n A_i x_i$$

para $x \in \mathbb{R}^n$. $B(x)$ es, claramente, simétrica. Queremos determinar si acaso existe un vector x tal que la matriz $B(x)$ sea semidefinida positiva, y en ese caso, queremos obtener el x con menor norma euclídeana que cumpla esa condición.

El modelo de optimización que resuelve el problema sería:

$$\begin{aligned} \text{mín} \quad & \|x\|_2 \\ \text{s.a.} \quad & B = A_0 + \sum_{i=1}^n A_i x_i \\ & B \succeq 0 \end{aligned}$$

donde $B \succeq 0$ denota pertenencia al cono de matrices simétricas y semidefinidas positivas.

Ahora, este problema puede ser transformado en:

$$\begin{aligned} \text{mín} \quad & t \\ \text{s.a.} \quad & B = A_0 + \sum_{i=1}^n A_i x_i \\ & \|x\|_2 \leq t \\ & B \succeq 0 \end{aligned}$$

que se escribe, en forma cónica:

$$\begin{aligned} \text{mín} \quad & t \\ \text{s.a.} \quad & B = A_0 + \sum_{i=1}^n A_i x_i \\ & (x, t) \in K^{n+1} \\ & B \succeq 0 \end{aligned}$$

donde K^{n+1} denota el cono de segundo orden.

Ahora mostremos cuál es la barrera asociada a este problema. Como siempre, la barrera se aplica a los conos y desigualdades. En este caso, al cono de segundo orden y al cono SDP, y es:

$$\Phi(x) = -\{\log(t^2 - x^T x) + \log(\det(B))\}$$

Recordemos que, para una matriz simétrica y definida positiva B ,

$$\det(B) = \prod_{j=1}^n \lambda_j$$

donde los λ_i son los valores propios de B . Entonces, $\log(\det(B)) = \sum_{j=1}^n \log \lambda_j$, así que la expresión es consistente con lo que habíamos mostrado en el ejemplo de la elipse de menor volumen.

9.7.7. Ejemplo 7: Problema cuadrático

Consideremos el siguiente problema:

$$\begin{aligned} \text{mín} \quad & \|u\|_2^2 + \|v\|_2^2 \\ \text{s.a.} \quad & Ax + u = b \\ & Bx + v \leq d \\ & \sum_{j=1}^n x_j = 1 \\ & x \in \mathbb{R}^n \end{aligned}$$

donde $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{p \times n}$, $b \in \mathbb{R}^m$, $d \in \mathbb{R}^p$. Además, se sabe que n es bastante grande.

Vamos a realizar una serie de transformaciones para llevar el problema a una formulación cónica. El problema podría ser transformado a la siguiente forma equivalente:

$$\begin{aligned} \text{mín} \quad & \sqrt{\|u\|_2^2 + \|v\|_2^2} \\ \text{s.a.} \quad & Ax + u = b \\ & Bx + v \leq d \\ & \sum_{j=1}^n x_j = 1 \\ & x \in \mathbb{R}^n \end{aligned}$$

Esto es válido, dado que la función raíz es creciente. Ahora, podemos transformarlo a:

$$\begin{aligned} \text{mín} \quad & t \\ \text{s.a.} \quad & Ax + u = b \\ & Bx + v \leq d \\ & \sum_{j=1}^n x_j = 1 \\ & \sqrt{\|u\|_2^2 + \|v\|_2^2} \leq t \\ & x \in \mathbb{R}^n \end{aligned}$$

que es lo mismo que:

$$\begin{aligned} \text{mín} \quad & t \\ \text{s.a.} \quad & Ax + u = b \\ & Bx + v \leq d \\ & \sum_{j=1}^n x_j = 1 \\ & \sqrt{u^T u + v^T v} \leq t \\ & x \in \mathbb{R}^n \end{aligned}$$

y este problema tiene el formato de problema lineal con restricciones de cono de segundo orden:

$$\begin{aligned} \text{mín} \quad & t \\ \text{s.a.} \quad & Ax + u = b \\ & Bx + v \leq d \\ & \sum_{j=1}^n x_j = 1 \\ & (u, v, t) \in Q^{m+p+1} \\ & x \in \mathbb{R}^n \end{aligned}$$

lo que puede ser resuelto con algoritmos de punto interior.

Desarrollamos a continuación el problema penalizado que aparecerá en cada iteración del algoritmo de punto interior. Hay que considerar función de barrera para todas las desigualdades e inclusiones cónicas. Denotemos por $\beta_1, \dots, \beta_p$ las filas de la matriz B . La barrera es:

$$\Phi(x, u, v, t) = - \sum_{i=1}^p \log(d_i - \beta_i^T x + v_i) - \log(t^2 - u^T u - v^T v)$$

y el problema penalizado, para un parámetro de penalización $\mu > 0$ es:

$$\begin{aligned} \text{mín} \quad & t + \mu \Phi(x, u, v, t) \\ \text{s.a.} \quad & Ax + u = b \\ & \sum_{j=1}^n x_j = 1 \\ & x \in \mathbb{R}^n \end{aligned}$$

9.7.8. Ejemplo 8: Otro problema cuadrático

Consideremos el siguiente problema:

$$P) \quad \begin{array}{ll} z^* = \text{mín} & \sqrt{x^T Q x} + c^T x \\ \text{s.a.} & Ax \leq b \\ & x \geq 0 \end{array}$$

donde $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$, $Q \in \mathbb{R}^{n \times n}$ y es simétrica y definida positiva.

Podemos transformar el problema a la siguiente forma:

$$P) \quad \begin{array}{ll} z^* = \text{mín} & t + c^T x \\ \text{s.a.} & Ax \leq b \\ & \sqrt{x^T Q x} \leq t \\ & x \geq 0 \end{array}$$

Ahora, este problema se puede dejar en forma de formulación cónica “pura” notando que como Q es simétrica y definida positiva, existe una matriz R tal que $Q = R^T R$ (la que viene de la descomposición de Cholesky, por ejemplo). Tenemos, entonces, que $\sqrt{x^T Q x} \leq t$ equivale a $\sqrt{x^T R^T R x} \leq t$. Llamemos $u = Rx$, tenemos que la restricción es ahora igual a $\sqrt{u^T u} \leq t$, que es el formato cónico de segundo orden. El problema es:

$$P) \quad \begin{array}{ll} z^* = \text{mín} & t + c^T x \\ \text{s.a.} & Ax \leq b \\ & \sqrt{u^T u} \leq t \\ & u = Rx \\ & x \geq 0 \end{array}$$

que, en forma cónica, es:

$$P) \quad \begin{array}{ll} z^* = \text{mín} & t + c^T x \\ \text{s.a.} & Ax \leq b \\ & (u, t) \in Q^{n+1} \\ & u = Rx \\ & x \geq 0 \end{array}$$

donde Q^{n+1} denota el Cono de Segundo Orden de dimensión $n + 1$.

9.7.9. Ejemplo 9: Restricciones hiperbólicas

Supongamos que tenemos restricciones en las variables x, y, z de la siguiente forma:

$$xy \geq z^2, \quad x \geq 0, \quad y \geq 0.$$

En una primera mirada, esto se ve extremadamente no lineal. Sin embargo, podemos reescribir esto en un formato de Cono de Segundo Orden como: We can model this with the second-order cone as:

$$(x - y, 2z, x + y) \in Q^3.$$

En efecto, escribamos lo anterior de la siguiente forma:

$$\sqrt{(x - y)^2 + (2z)^2} \leq x + y.$$

Elevando al cuadrado en ambos lados se obtiene:

$$x^2 - 2xy + y^2 + 4z^2 \leq x^2 + 2xy + y^2.$$

Reordenando y dividiendo por 4 se obtiene $xy \geq z^2$. También notemos que se tiene, de la restricción de cono, que $x + y \geq |x - y|$, lo que implica que $x + y \geq -x + y$ (y, por lo tanto, $x \geq 0$) y también $x + y \geq x - y$ (y, por lo tanto, $y \geq 0$).

9.7.10. Ejemplo 9: Productos de variables no negativas

Mostramos en este ejemplo una forma más general de modelar productos de variables no negativas como SOCP. Consideremos el siguiente ejemplo en variables $x_1, x_2, \dots, x_{16}$ y t :

$$x_1 x_2 x_3 \cdots x_{16} \geq t^{16}, \quad x_1 \geq 0, x_2 \geq 0, \dots, x_{16} \geq 0. \quad (9.1)$$

Vamos a agregar $14 = 16 - 2$ variables adicionales $y_1, \dots, y_8, z_1, \dots, z_4$, y w_1, w_2 , y generar las siguientes $15 = 16 - 1$ restricciones:

$$\begin{array}{llllll} x_1 x_2 & \geq & y_1^2 & & & \\ x_3 x_4 & \geq & y_2^2 & & & \\ x_5 x_6 & \geq & y_3^2 & y_1 y_2 & \geq & z_1^2 \\ x_7 x_8 & \geq & y_4^2 & y_3 y_4 & \geq & z_2^2 \\ x_9 x_{10} & \geq & y_5^2 & y_5 y_6 & \geq & z_3^2 \\ x_{11} x_{12} & \geq & y_6^2 & y_7 y_8 & \geq & z_4^2 \\ x_{13} x_{14} & \geq & y_7^2 & & z_1 z_2 & \geq & w_1^2 \\ x_{15} x_{16} & \geq & y_8^2 & & z_3 z_4 & \geq & w_2^2 \\ & & & & & & w_1 w_2 \geq t^2 \end{array}$$

en variables no negativas

$$x_1 \geq 0, x_2 \geq 0, \dots, x_{16} \geq 0, y_1 \geq 0, \dots, y_8 \geq 0, z_1 \geq 0, \dots, z_4 \geq 0, w_1 \geq 0, w_2 \geq 0.$$

Notemos que cada una de las 15 restricciones hiperbólicas puede ser reescrita como SOCP usando la construcción del ejemplo anterior. Notemos, también, que esas restricciones implican que:

$$x_1 x_2 x_3 \cdots x_{16} \geq (y_1 y_2 y_3 \cdots y_8)^2 \geq (z_1 z_2 z_3 z_4)^4 \geq (w_1 w_2)^8 \geq t^{16}.$$

Inversamente, si $x_1, x_2, \dots, x_{16}$ y t satisfacen (9.1), entonces se pueden construir valores de $y_1, y_2, \dots, y_8, z_1, z_2, z_3, z_4$, y w_1, w_2 para los cuales el sistema de desigualdades anterior se satisface.